Task Analysis

Today's Topics

- User analysis
- Task analysis
- Domain analysis (context)
- Requirements analysis

Know Your User

- Identify characteristics of target user population
  - Age, gender, culture, language
  - Education (literacy? numeracy?)
  - Physical limitations
  - Computer experience (typing?)
  - Motivation, attitude
  - Domain experience
  - Application experience
  - Work environment and other social context
  - Relationships and communication patterns

Multiple Classes of Users

- Many applications have several kinds of users
  - By role (student, teacher)
  - By characteristics (age, motivation)
- Example: Olympic Message System
  - Athletes
  - Friends & family
  - Telephone operators
  - Sysadmins

Personas

- A persona is a fictitious character used as a specific representative of a user class
  - Yoshi is a 20-year-old pole vaulter from Tokyo who speaks some English
  - Bob is an IBM sysadmin in New York
  - Fritz is the 50-year-old father of a German swimmer
- Advantages
  - Convenient handle for talking about user classes
  - Focuses on a typical user, rather than an extreme
  - Encourages empathy
- Disadvantages
  - May be misleading
  - Stereotype trap

Franny is an 8-year old child (typical of some user class), who happens to be a girl, lives in Chicago, and likes drawing bunnies and mushroom clouds (but those aren't typical for the class, nor do they really matter for design)

How To Do User Analysis

- Techniques
  - Questionnaires
  - Interviews
  - Observation
- Obstacles
  - Developers and users are sometimes systematically isolated from each other
    - Tech support shields developers from users
    - Marketing shields users from developers
  - Some users are expensive to talk to
    - Doctors, executives, union members

Task Analysis

- Identify the individual tasks the program might solve
- Each task is a goal (*what*, not *how*)
- Often helps to start with overall goal of the system and then decompose it hierarchically into tasks

Essential Parts of Task Analysis

- What needs to be done?
  - Goal
- What must be done first to make it possible?
  - Preconditions
    - Tasks on which this task depends
    - Information that must be known to the user
- What steps are involved in doing the task?
  - Subtasks
  - Subtasks may be decomposed recursively

Example from OMS

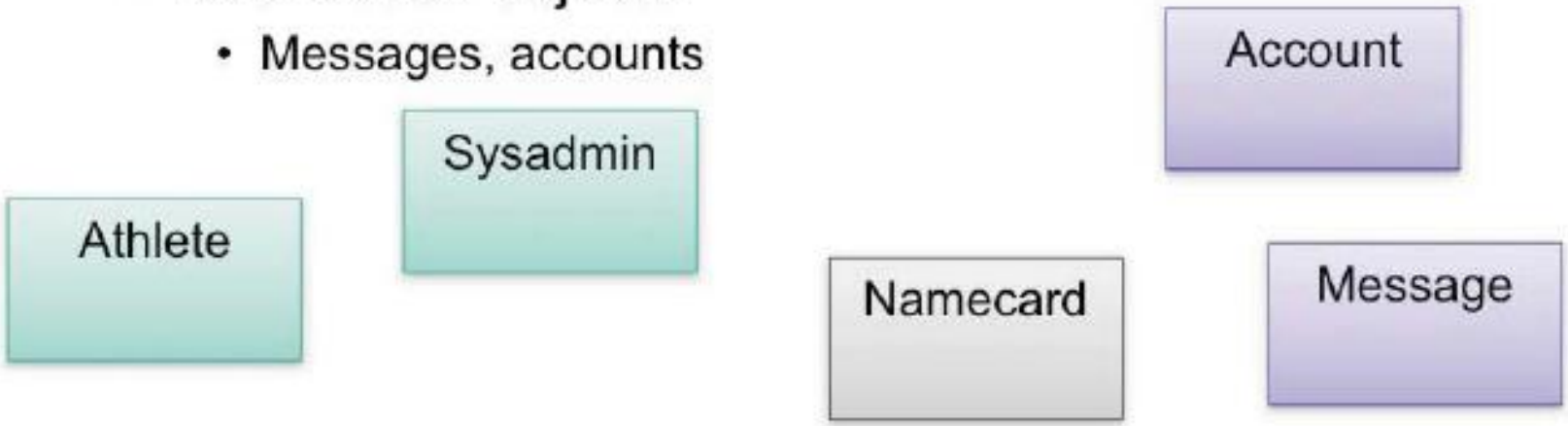
- Goal
  - Send message to another athlete
- Preconditions
  - Must know: my country code, my username, my password, the other athlete's name
- Subtasks
  - Log in (identify yourself)
  - Identify recipient
  - Record message
  - Hang up

Other Questions to Ask About a Task

- Where is the task performed?
  - At a kiosk, standing up
- What is the environment like? Noisy, dirty, dangerous?
  - Outside
- How often is the task performed?
  - Perhaps a couple times a day
- What are its time or resource constraints?
  - A minute or two (might be pressed for time!)
- How is the task learned?
  - By trying it
  - By watching others
  - Classroom training? (probably not)
- What can go wrong? (Exceptions, errors, emergencies)
  - Enter wrong country code
  - Enter wrong user name
  - Get distracted while recording message
- Who else is involved in the task?

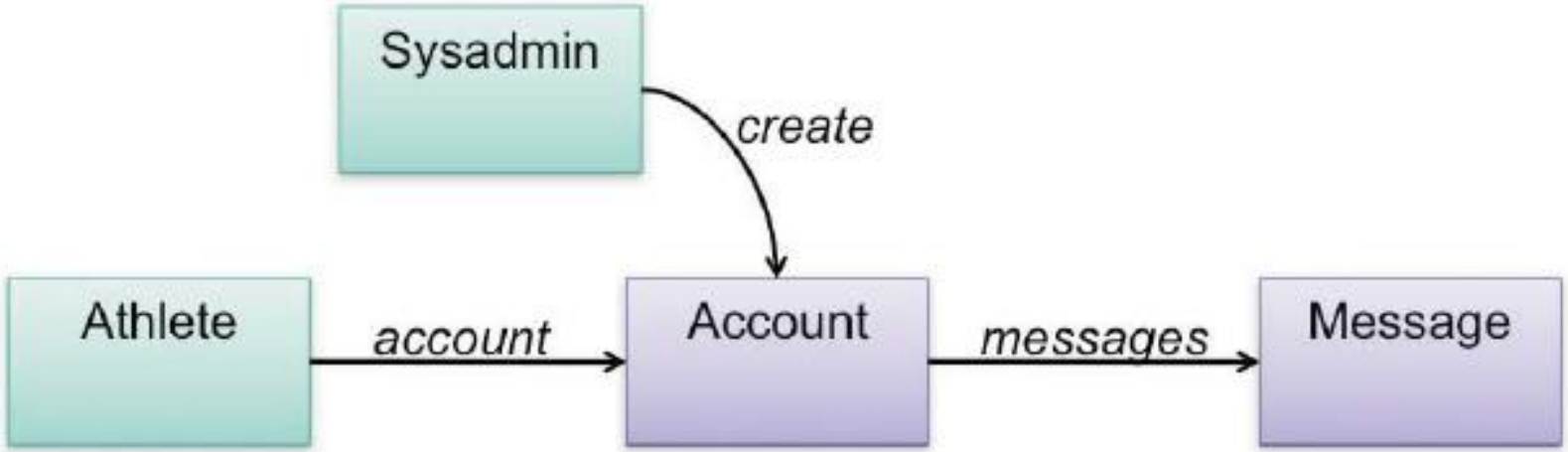
Domain Analysis

- Identify important things in the domain
  - People (user classes)
    - Athletes, friends & family, sysadmins
  - Physical objects
    - Namecard, telephone
  - Information objects
    - Messages, accounts



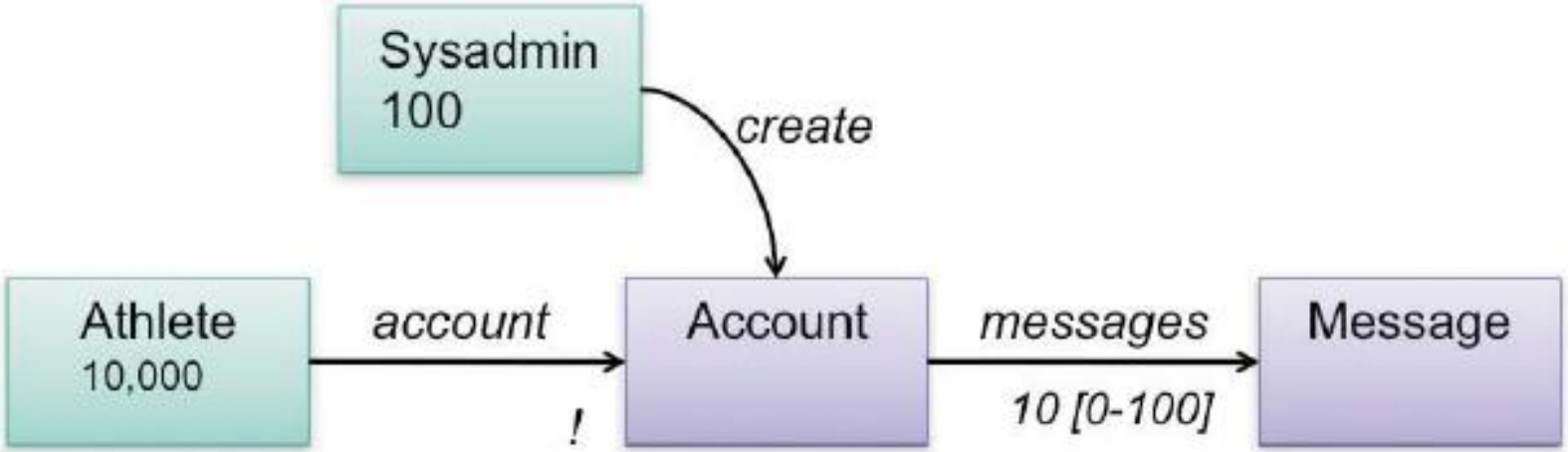
Domain Analysis

- Determine important relations between the things
  - Athletes *have* accounts
  - Accounts *have* messages
  - Family & friends *know* athletes
  - Sysadmins *register* athletes or *create* accounts



Domain Analysis

- Identify multiplicities of things and relations
  - Numbers are best, but simple multiplicity indicators (!,?,+,*) help too



10,000 messages must be displayed and manipulated much differently than 10 messages

! means exactly 1, ? means 0 or 1, + means 1 or more, and * means 0 or more

- Missing tasks
  - CRUD: Create, Read, Update, Delete

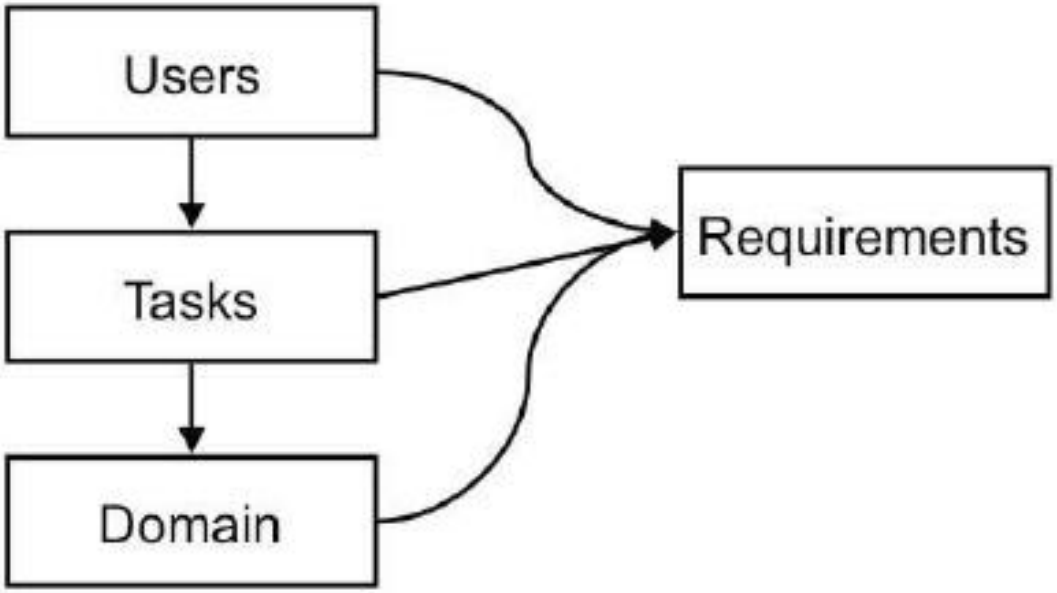
One heuristic that you can use for information objects

Messages in the OMS example:

friends and family need to record messages (Create),
athletes need to listen to them (Read),
athletes need to delete them (Delete).
We’re missing the Update task.

Requirements Analysis

- Requirements: what should the system do?



Common Errors in User Analysis

- Describing what your ideal users *should* be, rather than what they actually *are*
  - "Users should be literate in English, fluent in spoken Swahili, right-handed, and color-blind"
- Thinking from the system's point of view, rather than the user's
  - "Notify user about appointment"
  - vs. "Get a notification about appointment"
- Fixating too early on a UI design vision
  - "The system bell will ring to notify the user about an appointment..."
- Bogging down in *what* users do now (**concrete** tasks), rather than *why* they do it (**essential** tasks)
  - "Save file to disk"
  - vs. "Make sure my work is kept"
- Duplicating a bad existing procedure in software
- Failing to capture good aspects of existing procedure

A dentist's office converted from manual billing to an automated system. But the assistants didn't like the new system, because they were accustomed to keeping important notes on the paper forms, like "this patient's insurance takes longer than normal." Automated system provided no way to capture those kinds of annotations

Hints for Better User & Task Analysis

- Questions to ask
  - Why do you do this? (goal)
  - How do you do it? (subtasks)
- Look for weaknesses in current situation
  - Goal failures, wasted time, user irritation
- Contextual inquiry
- Participatory design

Contextual Inquiry

- Observe users doing real work in the real work environment
- Be concrete
- Establish a master-apprentice relationship
  - User shows how and talks about it
  - Interviewer watches and asks questions
- Challenge assumptions and probe surprises

Participatory Design

- Include representative users directly in the design team
- OMS design team included an Olympic athlete as a consultant

particularly vital when the target users have much deeper domain knowledge than the design team

Summary

- User analysis identifies the user classes
- Task analysis discovers their tasks
- Domain analysis finds the entities and relationships in the domain